

Introduction

Un algorithme est un ensemble fini d'ordre (instructions) accomplissant une tâche bien déterminée

L'algorithmique désigne le processus de recherche de l'algorithme

Pour pouvoir écrire des algorithmes => il existe des conditions :

- 1- condition d'entrée (cad il existe 0 ou plusieurs données d'entrée)
- 2- condition de sortie (pour tout algorithme il doit y avoir au moins un résultat)
- 3- condition de bonne définition (une instruction doit être claire et non ambiguë)
- 4- condition de terminaison d'un algorithme (tout algorithme doit terminer dans un nombre fini d'étapes)
- 5- condition de réalisation (toute instruction doit être suffisamment de base (peut s'exécuter indépendamment des autres))
 - a- Besoins du problème (comprendre les exigences du problème)
 - b- Conception d'un algorithme
 - c- **Analyse (existe-t-il un algorithme meilleur ?)**
 - d- Raffinement et codage (écriture de l'algorithme en programme tout en choisissant la structure de données adéquate)
 - e- Vérification d'un programme qui comprend 3 phases :
 - preuve du programme (est ce que le programme répond correctement aux données)
 - test
 - debugging (dans le cas de tests erronés)

Un bon algorithme est un algorithme :

- correct : exécution correctes des tâches pour lesquelles il a été conçu
- complet : il faut que le programme considère tous les cas possibles et donne un résultat dans chaque cas,
- efficace : exécution avec un coût minimum (coût par rapport à l'ordinateur)
temps de calcul + espace nécessaire

But : - Elaborer des algorithmes performants et efficaces

- connaître la notion de complexité
- connaître la complexité des différents types d'algorithmes (itératifs, récursifs, gloutons,

Comment Analyser ?

- ⇒ Critère temps d'exécution d'une instruction simple (dépend de la machine, langage, compilateur) → complexité temporelle
- ⇒ Critère espace mémoire (la fréquence d'exécution d'une instruction donnée dans un algorithme → complexité spatiale

Exemples :

1-Calcul du nombre d'exécution de l'instruction $A=A*5$

ALGO 1	ALGO 2	ALGO3
- - $A=A*5$ - -	- - Pour i de 1 à n faire $A=A*5$ Fin pour - -	- - Pour i de 1 à n faire Pour j de 1 à n faire $A=A*5$ Fin pour Fin pour

ALGO 4	ALGO 5	ALGO 6
- - Pour i de 1 à n faire $A=A*5$ Pour j de 1 à k faire $A=A*5$ Fin pour Fin pour	- - Pour i de 1 à n faire $A=A*5$ Fin pour Pour j de 1 à k faire $A=A*5$ Fin pour	- - Pour i de 1 à n faire Pour j de 1 à i faire $A=A*5$ Fin pour Fin pour

ALGO 1	ALGO 2	ALGO 3	ALGO 4	ALGO 5	ALGO 6
1	n	n^2	$n(1+k)$	$n+k$	$n(n+1)/2$

2- Calcul du Nième terme de la suite de Fibonnachi ($N \geq 0$)

ALGORITHME
<pre>Lire (N) Si (N<=1) Alors Ecrire (N) Sinon F0=0 F1=1 Pour i de 2 à N faire F=F1+F0 F0=F1 F1=F Fin pour Ecrire (F) Finsi Fin algorithme</pre>

Nombre de Comparaisons	Nombre d'Affectations	Nombre d'Affichages
1 N (pour)	2 N (pour) 3*(N-1) (traitement)	1
1+N	2+N+3*(N-1)	1

D'une manière générale T est fonction (du nombre de comparaisons, d'affectations, des affichages,...)

Notion de Complexité

En informatique le mot complexité s'articule autour de

La complexité algorithmique

C'est l'étude de l'efficacité comparée des algorithmes. On cherche à analyser le comportement d'un algorithme en mesurant son temps d'exécution ainsi que l'espace mémoire nécessaire. Ceci peut se faire d'une manière formelle ou expérimentale.

La complexité des problèmes

Dans ce cas on s'intéresse à l'étude du meilleur algorithme permettant de résoudre un problème donné. La complexité des algorithmes a permis de classer des problèmes en fonction des performances des algorithmes qui les résolvent. Cette classification a été conçue de manière indépendante des caractéristiques techniques des ordinateurs ($\forall$ évolution des ordinateurs <-----> classification inchangée).

Complexité algorithmique

Pour calculer la complexité d'un algorithme il est nécessaire de :

- Connaître la taille des entrées ou encore instances :

La taille n dépend du codage de ces entrées.

Exple en binaire il faut $(\log_2(n) + 1)$ bits pour coder l'entier n .

En pratique on choisit comme taille la ou les dimensions les plus significatives.

Quelques exemples, selon la modélisation du problème par :

Des nombres	Ces nombres
Des polynômes	Le degré, le nombre de coefficients
Des matrices $m \times n$	Max(m, n), m, n , $m+n$, ($m \times n$)
Des graphes	Ordre, taille, produit des deux
Des arbres	Idem que graphes +, hauteur, l'arité
Des mots	Leur longueur

Le choix d'une structure de données n'est pas sans effet sur l'efficacité d'un algorithme.

Structure de données	statique	taille mémoire fixe /allouée (tableau)
	Dynamique	pile, fichier, liste chaînée => taille variable

- Repérer les opérations fondamentales :

Au niveau d'un algorithme certaines opérations sont plus importantes (coût) que d'autres. C'est autour de ces opérations que l'évaluation de l'algorithme se fera.

La complexité d'un algorithme est la mesure du nombre d'opérations fondamentales qu'il effectue sur un jeu de données. Elle est souvent déterminée à travers une description mathématique du comportement de cet algorithme.

Exemple	Opération(s) fondamentale(s)
Recherche d'un élément Dans un tableau, une liste, un arbre	Comparaisons
Tri d'une liste, d'un tableau, d'un fichier	Comparaisons, déplacements
Multiplication Des polynômes, des matrices, de grands entiers	Additions, multiplications

- Evaluer le coût des opérations

D'une manière générale les opérations n'ont pas toutes le même coût.

Exemple Coût (*) >> Coût (+)

Plusieurs notations sont utilisées pour déterminer la complexité des algorithmes

O : Borne supérieure asymptotique

Ω : Borne inférieure asymptotique

Θ : Borne asymptotique

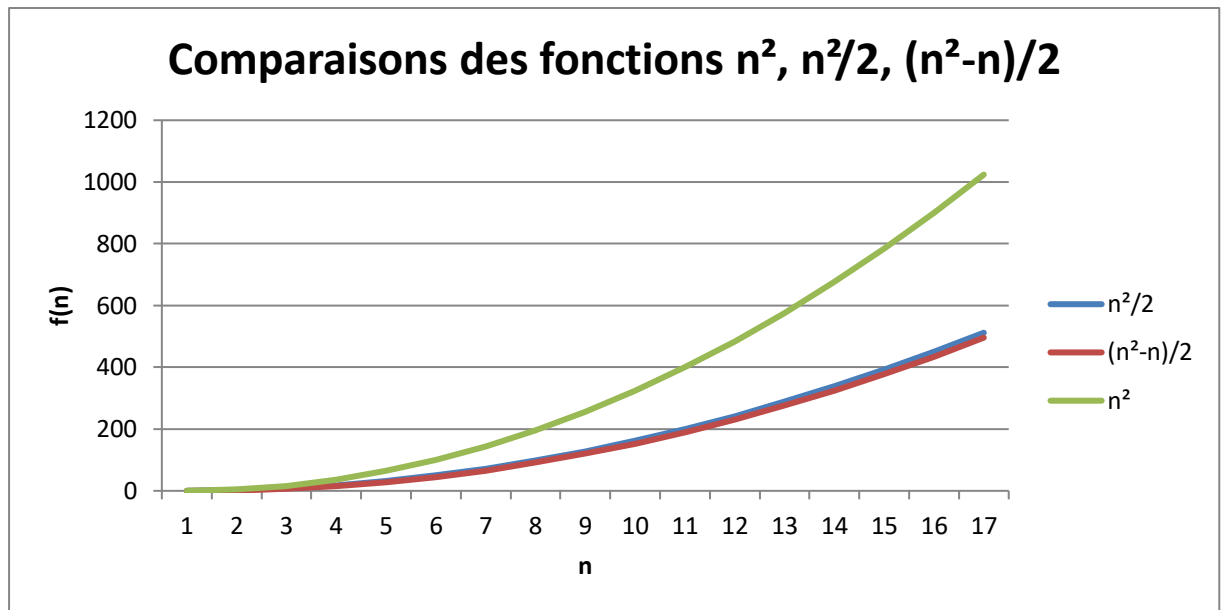
Définition a : Borne supérieure asymptotique O

$O(g(n)) = \{f: N \rightarrow N \mid \exists k > 0 \text{ et } n_0 \geq 0 \text{ tels que } \forall n \geq n_0, 0 \leq f(n) \leq k g(n)\}$

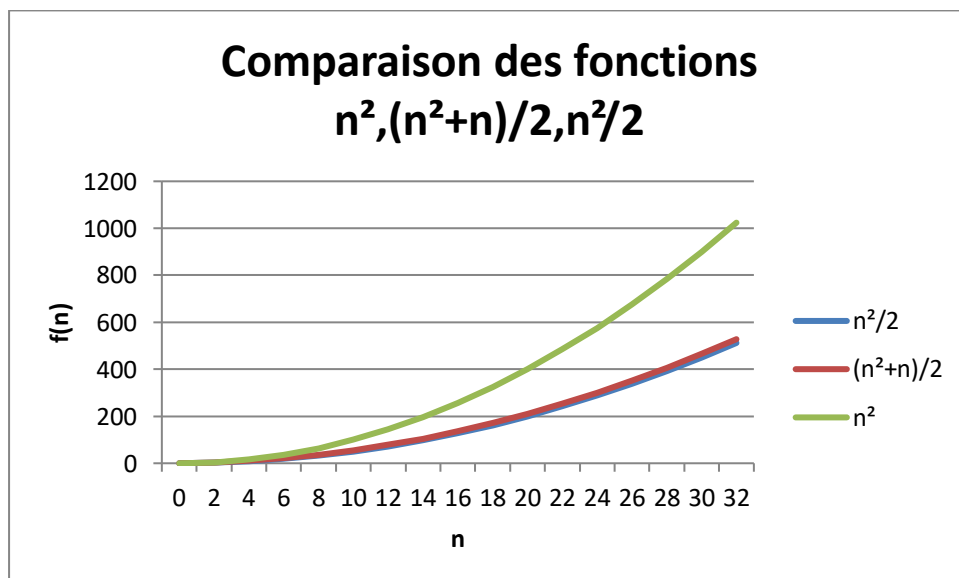
Si une fonction $f(n) \in O(g(n))$, on dit que $g(n)$ est une borne supérieure asymptotique pour $f(n)$. On note $f(n) = O(g(n))$.

C'est la plus petite borne supérieure qui est cherchée

On considère $f(n) = 1/2(n^2 - n)$, $f(n)$ est approximé à $O(n^2)$ car pour $c = 1/2$ et $n_0 = 0$ on a $\forall n \geq 0, 1/2(n^2 - n) \leq 1/2 n^2$



On considère $f(n) = an^2 + bn$; $an^2 + bn \leq an^2 + bn^2 \leq (a+b)n^2 \quad \forall n \geq 1$
 $f(n) \leq cn^2$ où $c = a + b$ et $n_0 = 1 \Rightarrow f(n)$ est approximée à $O(n^2)$

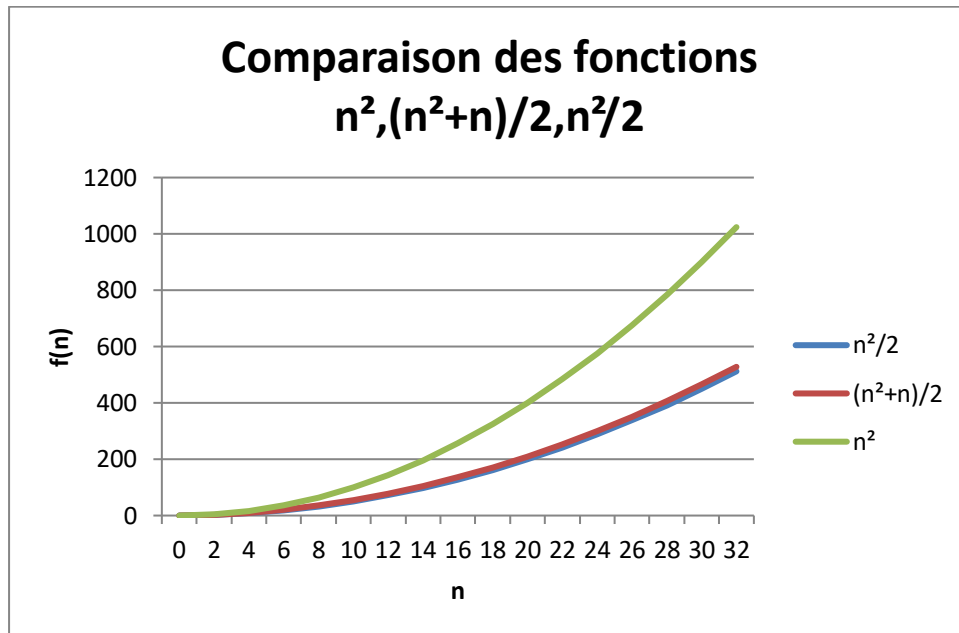


Définition b : Borne supérieure asymptotique Ω

$\Omega(g(n)) = \{f: N \rightarrow N \mid \exists k > 0 \text{ et } n_0 \geq 0 \text{ tels que } \forall n \geq n_0 \quad 0 \leq kg(n) \leq f(n)\}$
 Si une fonction $f(n) \in \Omega(g(n))$, on dit que $g(n)$ est une borne inférieure asymptotique pour $f(n)$. On note $f(n) = \Omega(g(n))$.

C'est la plus grande borne inférieure qui est cherchée

On considère $f(n) = an^2 + bn$; $an^2 + bn \geq an^2 \quad \forall n > 1$
 $f(n) \geq cn^2$ où $c = a$ et $n_0 = 1$ $f(n)$ est approximée à $O(n^2)$



Définition c : Borne asymptotique Θ

Lorsque le O et le Ω d'une fonction coïncident on utilise la notation Θ

$\Theta(g(n)) = \{f: N \rightarrow N \mid \exists k_1 > 0, \exists k_2 > 0 \text{ et } n_0 \geq 0 \text{ tels que } \forall n \geq n_0 \ 0 \leq k_1 g(n) \leq f(n) \leq k_2 g(n)\}$

Si une fonction $f(n) \in \Theta(g(n))$, on dit que $g(n)$ est une borne asymptotique pour $f(n)$.

On note $f(n) = \Theta(g(n))$.

Notation mathématique O

La notation mathématique O est la plus utilisée pour expliquer formellement la performance d'un algorithme. Elle reflète la courbe de croissance de celui-ci. Les règles de la notation O sont :

$$O(c) = O(1)$$

$$O(cT) = cO(T) = O(T)$$

$$O(T_1) + O(T_2) = O(T_1 + T_2) = \max(O(T_1), O(T_2))$$

$$O(T_1) \cdot O(T_2) = O(T_1 \cdot T_2)$$

Exemple : $T(n) = 5n^2 + 7n + 10$

$$O(T(n)) = O(n^2)$$

$$n=10 \quad 5n^2/T(n) = 5 \cdot 100 / (5 \cdot 100 + 70 + 10) = 50/580 \quad 86\%$$

$$7n/T(n) = 70/580 = 0,12 \quad 12\%$$

$$10/580 = 0,018 \quad 1,8\%$$

Dans notre contexte on supposera que toutes les opérations ont un coût uniforme (constant) dénoté $O(1)$.

- Evaluation des coûts en séquentiel

- Dans un algorithme séquentiel il n'y a pas de récursivité. (Dans ce cas les boucles sont disjointes ou emboîtées imbriquées). L'exécution des instructions s'effectue une à une de haut vers le bas. Les temps d'exécution s'additionnent :

Séquence d'instructions simples Dans le cas où on a une série d'instructions en $O(1)$,	... Ins 1 $O(1)$ Ins 2 $O(1)$ Ins 7 $O(1)$ la somme l'est aussi (cad en $O(1)$) $\Rightarrow O(1)$
Alternative	Si cond Alors B1 Sinon B2 Fin Si $T(n) = T_{cond}(n) + \text{Max}(TB1(n), TB2(n))$

Itération Bornée	Pour i de j à k faire B Fin pour $T(n) = (k-j+1) * (T_{ind}(n) + TB_n) + T_{ind}(n)$ Tind est relatif à l'affectation de l'indice de boucle et le test de continuation
------------------	--

- Hypothèse : Le traitement à l'intérieur de la boucle est en $O(1)$
- Si le nombre d'itérations est constant alors la complexité est $O(1)$ (exple Pour i de 1 à 7 Faire)
- Si le nombre d'itérations est fixée à N alors la complexité est $O(N)$
Pour i de 1 à N faire
 qqe chose en $O(1)$ N opérations
Fin pour
- Si 2 boucles imbriquées / N itérations chacune
 Instructions en $O(1)$
complexité est $O(N^2)$
- Si 3 boucles imbriquées / N itérations chacune
 Instructions en $O(1)$
complexité est $O(N^3)$
- Si M boucles imbriquées / N itérations chacune
 Instructions en $O(1)$
complexité est $O(N^M)$

Pour i de 1 à N faire
 Pour j de 1 à N faire
 qqe chose en $O(1)$ N opérations
 Fin pour
 Fin pour
 N opérations à réaliser N fois ($N*N=N^2$)

Itération non bornée	Tant que cond faire B Fin tant que $T(n) = \text{nb_boucles} * (TB(n) + T_{\text{cod}}(n)) + T_{\text{cond}}(n)$ Répéter B Jusqu'à Cond $T(n) = \text{nb_boucles} * (TB(n) + T_{\text{cod}}(n))$ Le nombre de boucles nb_boucles s'évalue de manière inductive
----------------------	---

Exemple1

var = 1
 Tant que var ≤ n faire
 :
 qqe chose en $O(1)$
 var = 2 * var
 Fin tant que
 Complexité temporelle?

Var = cte * Var

cte = 2 Var = 1 = 2^0
 Itération 1 Var = 2 * Var = 2 = 2^1
 Itération 2 Var = 2 * Var = 2 * 2 = 2^2
 :
 Itération k Var = 2 * Var = $2 * 2^{k-1} = 2^k$
Déterminer la complexité temporelle => Trouver k (nombre d'itérations)
 Var = 2^k => $\log_2(\text{Var}) = \log_2(2^k) = k(\log_2(2)) = k$
 => $k = \log_2(\text{Var})$
 L'itération se termine quand Var ≥ N
Complexité temporelle $O(\log_2(N))$

Cas Général Var= cte* Var

$$\text{Var} = c^k \text{ Var}$$

$$\text{Var} = c^k \Rightarrow \log_c(\text{Var}) = \log_c(c^k) = k(\log_c(c)) = k$$

$$\Rightarrow k = \log_c(\text{var})$$

L'itération se termine quand $\text{Var} \geq N$

Pour $\text{Var} = N \Rightarrow k = \log_c(\text{var})$

Complexité temporelle $O(\log_c(N))$

Exemple 2

var = N

Tant que var > 1 faire

:

qqe chose en $O(1)$

var = var/2

Fin tant que

Complexité temporelle?

Var= Var/cte

cte=2 Var=N = $N/2^0$

Itération 1 Var = $\text{Var}/2 = N/2 = N/2^1$

Itération 2 Var = $\text{Var}/2 = N/2^2 = N/2^2$

:

Itération k Var = $\text{Var}/2 = N/2^{k-1} = N/2^k$

Déterminer la complexité temporelle $\Rightarrow$ Trouver k (nombre d'itérations)

$$\text{Var} = N/2^k \quad \log_2(\text{Var}) = \log_2(N/2^k) = \log_2(N) - \log_2(2^k)$$

$$= \log_2(N) - k(\log_2(2)) = \log_2(N) - k$$

$$k = \log_2(N) - \log_2(\text{Var})$$

L'itération se termine quand $\text{Var} \leq 1 \quad \log_2(1) = 0 \quad k \approx \log_2(N)$

Complexité temporelle $O(\log_2(N))$

Cas Général Var= Var/cte

$$\text{Var} = \text{Var}/c^k$$

$$\text{Var} = N/c^k$$

$$\log_c(\text{Var}) = \log_c(N/c^k) = \log_c(N) - k(\log_c(c)) = \log_c(N) - k$$

$$k = \log_c(N) - \log_c(\text{var})$$

L'itération se termine quand $\text{Var} \leq 1$

Pour $\text{Var} = 1 \quad k = \log_c(N)$

Complexité temporelle $O(\log_c(N))$

Appel de procédures	On peut ordonner les procédures de sorte que la ième ait sa complexité qui ne dépende que des procédures j/ j<i
---------------------	---

- Evaluation des coûts en récursif

Les algorithmes du type « diviser pour régner » donnent lieu à des programmes récursifs. Pour évaluer leur coût il faut trouver :

- La relation de récurrence associée
- La base de la récurrence
- La solution de base

Et ce, par résolution des équations de récurrence ou de partition

Exemples

Calcul de factoriel n

Opération fondamentale => * (multiplication)

Version itérative

r=1

Pour i de 1 à n Faire

r=r*i

Fin Pour

Complexité temporelle ? Complexité spatiale ?

Complexité temporelle

r=r*i O(1)
 n itérations $\Sigma O(i) = \Sigma O(1) = n$
 Complexité O(n)

Complexité spatiale O(1)

Réservation d'un espace mémoire pour **r** dont la valeur est mise à jour à chaque Itération.

Version récursive

Entier factrech (entier : n)

Début

Si n=0 retourne (1) ou factrech=1

Sinon

Retourne (n * factrech(n-1))

Fin Si

Fin

Complexité temporelle ? Complexité spatiale ?

Complexité temporelle

$$T(0)=0$$

$$T(n)=T(n-1)+1$$

$T(n)=T(n-1) + 1$ $T(n-1)=T(n-2) + 1$ $T(n-2)=T(n-3) + 1$ $\vdots$ $T(n-(n-1))=T(n-n) + 1$ $\Sigma(G)=\Sigma(D)$ $T(n)+T(n-1)+T(n-2)+\dots+ T(n-(n-1))$ $= T(n-1) + T(n-2) + T(n-3) + \dots +$ $T(n-n) + 1+1+1+\dots+1$ Après simplification $T(n)=T(0)+1+1+\dots+1=0+n=n$	Autre ment $T(n)=T(\cancel{n-1}) + 1$ $T(\cancel{n-1})=T(\cancel{n-2}) + 1$ $T(\cancel{n-2})=T(\cancel{n-3}) + 1$ $\vdots$ $T(\cancel{n-(n-1)})=T(n-n) + 1$	$T(n)= T(0)+1+1+\dots+1$ $T(n) = n$
---	--	-------------------------------------

Complexité $O(n)$

Complexité spatiale

$$n * \text{factrech}(n-1)$$

$$n-1 * \text{factrech}(n-2)$$

$$n-2 * \text{factrech}(n-3)$$

$$\vdots$$

$$1 * \text{factrech}(0)$$

$$1$$

Réservation d'un espace mémoire pour chaque valeur de la pile $n, n-1, n-2, \dots, 1$

Complexité spatiale $O(n)$

Recherche Dichotomique

Opération : comparaison

Tableau trié à n cases

$$T(n)=T(n/2) + 1$$

$$T(1)= 1$$

$$T(n) : O(\log_2(n))$$

On pose $n=2^k$

$T(2^k) = T(2^{k/2}) + 1 = T(2^{k-1}) + 1$ $T(2^{k-1}) = T(2^{k-2}) + 1$ $T(2^{k-2}) = T(2^{k-3}) + 1$ $\vdots$ $T(2^{k-(k-1)}) = T(2^{k-k}) + 1$	$T(2^k) = T(2^{k-1}) + 1$ $T(2^{k-1}) = T(2^{k-2}) + 1$ $T(2^{k-2}) = T(2^{k-3}) + 1$ $\vdots$ $T(2^{k-(k-1)}) = T(2^{k-k}) + 1 = 1 + 1$ $\Sigma(G) = \Sigma(D)$	$T(2^k) = 1 + 1 + \dots + 1 \text{ (k fois)} + 1$ $= k + 1 = \log_2(n) + 1$
---	---	--

Complexité $O(\log_2(n))$

Tour de Hanoï

$$T(n) = 2T(n-1) + 1$$

$$T(0) = 0$$

$$T(n) = 2^n - 1$$

$T(n) = 2T(n-1) + 1$ $T(n-1) = 2T(n-2) + 1$ $T(n-2) = 2T(n-3) + 1$ $\vdots$ $T(n-(n-1)) = 2T(n-n) + 1$	
$T(n) = 2T(n-1) + 1$ $2[T(n-1)] = 2[2T(n-2) + 1]$ $2^2[T(n-2)] = 2^2[2T(n-3) + 1]$ $\vdots$ $2^{n-1}[T(n-(n-1))] = 2^{n-1}[2T(n-n) + 1]$	
$T(n) = 2T(n-1) + 1 = 2T(n-1) + 2^0$ $2[T(n-1)] = 2[2T(n-2) + 1] = 2^2T(n-2) + 2^1$ $2^2[T(n-2)] = 2^2[2T(n-3) + 1] = 2^3T(n-3) + 2^2$ $\vdots$ $2^{n-1}[T(n-(n-1))] = 2^{n-1}[2T(n-n) + 1] = 2^nT(n-n) + 2^{n-1}$	$T(n) = 2T(n-1) + 2^0$ $2[T(n-1)] = 2^2T(n-2) + 2^1$ $2^2[T(n-2)] = 2^3T(n-3) + 2^2$ $\Rightarrow \vdots$ $2^{n-1}[T(n-(n-1))] = 2^nT(n-n) + 2^{n-1}$
$\Sigma(G) = \Sigma(D)$	$\Sigma(G) = \Sigma(D)$
$T(n) + 2[T(n-1)] + \dots + 2^{n-1}[T(n-(n-1))] =$ $2T(n-1) + 1 + 2[2T(n-2) + 1] + 2^{n-1}[2T(n-n) + 1]$ $T(n) = 2^nT(n-n) + 2^0 + 2^1 + 2^2 + \dots + 2^{n-1}$ $T(n) = 2^0 + 2^1 + 2^2 + \dots + 2^{n-1}$ $T(n) = 1(1-2^n)/(1-2) = 1(2^n-1) = 2^n-1$	$T(n) = 2T(n-1) + 2^0$ $2[T(n-1)] = 2^2T(n-2) + 2^1$ $2^2[T(n-2)] = 2^3T(n-3) + 2^2$ $\vdots$ $2^{n-1}[T(n-(n-1))] = 2^nT(n-n) + 2^{n-1}$ $T(n) = 2^nT(n-n) + 2^0 + 2^1 + 2^2 + \dots + 2^{n-1}$ $T(n) = 2^0 + 2^1 + 2^2 + \dots + 2^{n-1}$ $T(n) = 1(1-2^n)/(1-2) = 1(2^n-1) = 2^n-1$

Variantes du temps d'exécution

On évalue le temps d'exécution par rapport au pire des cas (max d'opérations*), en moyenne des cas et au meilleur cas (min d'opérations*)

* Opérations jugées fondamentales relatives au traitement

Temps d'exécution Pire des cas	$T_{pire}(n) = \max_n \{T(n)\}$
Temps d'exécution cas moyen	$T_{moy}(n) = \sum_n p(n) * T(n)$ dans la cas d'une loi de probabilité uniforme => $T_{moy_unif}(n) = (1/\#n) \sum_n T(n)$
Temps d'exécution meilleur cas	$T_{meill}(n) = \min_n \{T(n)\}$

Ordre de complexité

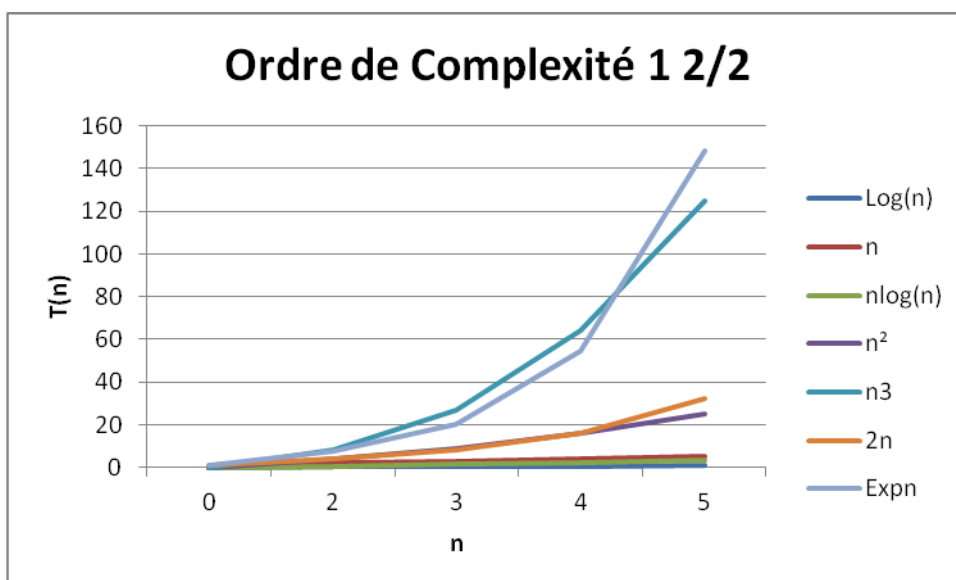
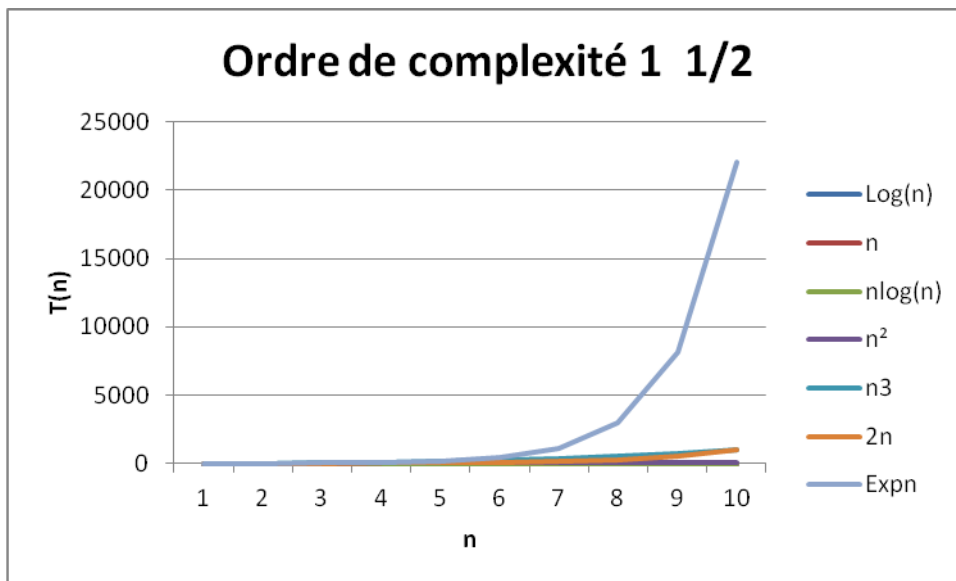
L'ordre de complexité du plus petit au plus grand

$O(1)$, $O(\text{Log}(n))$, $O(n^x)$, $O(n)$, $O(n \text{Log}(n))$, $O(n^2)$, $O(c^n)$, $O(n!)$ avec $0 < x < 1 < c$

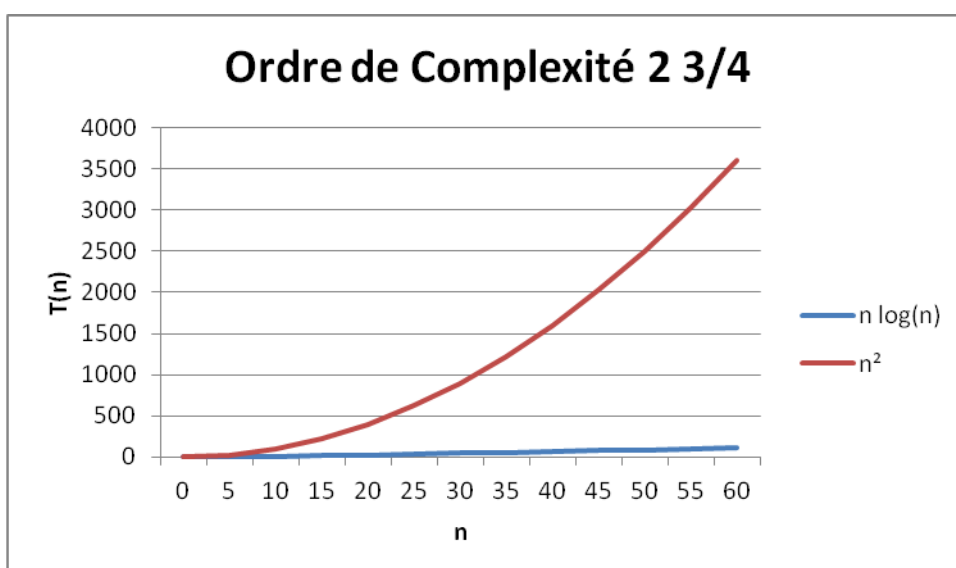
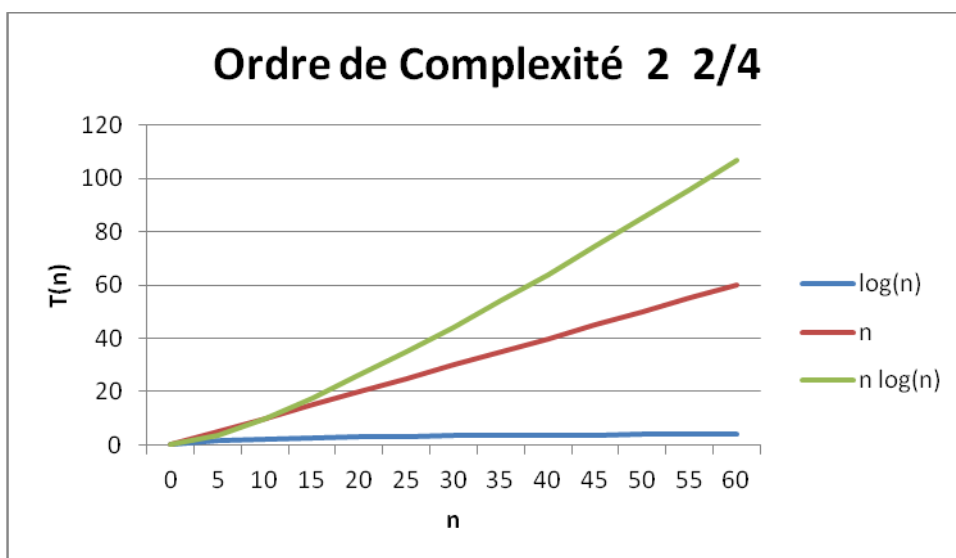
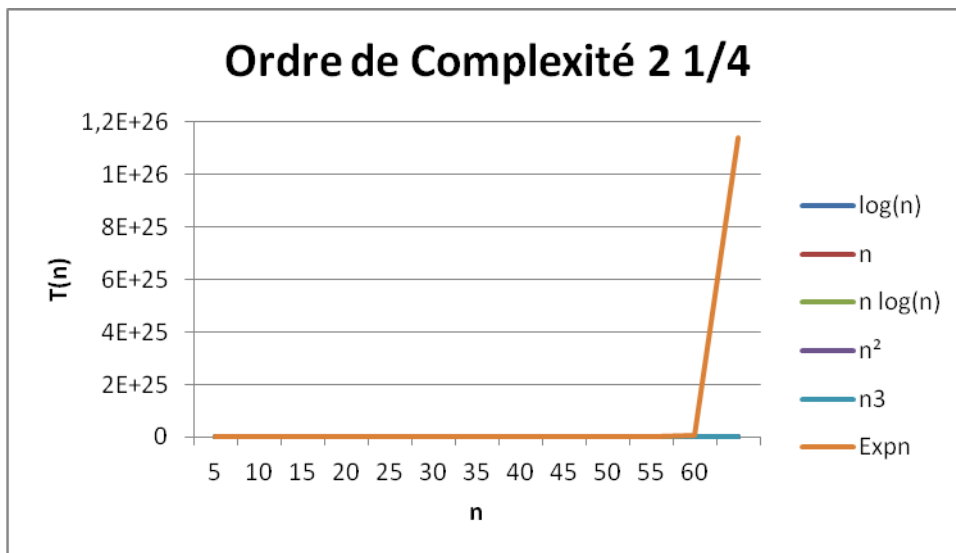
$O(1)$	Constant
$O(\text{Log}(n))$	logarithmique (quand n augmente => meilleur ordre)
$O(n)$	Linéaire
$O(n \text{Log}(n))$	$n \text{Log}(n)$
$O(n^2)$	Quadratique
$O(n^3)$	Cubique
$O(2^n)$	Exponentiel

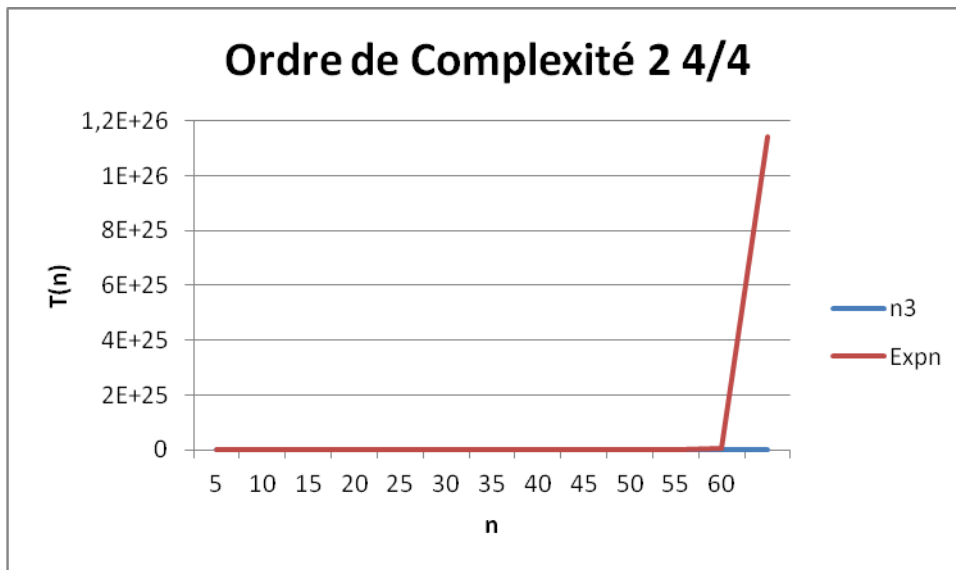
Un algorithme est polynomial s'il existe un entier p tel que son coût soit en $O(n^p)$

	$\text{Log}(n)$	n	$n \text{log}(n)$	n^2	n^3	2^n	Exp^n
0	-	0	-	0	0	1	1
2	0,30103	2	0,60205999	4	8	4	7,3890561
3	0,47712125	3	1,43136376	9	27	8	20,0855369
4	0,60205999	4	2,40823997	16	64	16	54,59815
5	0,69897	5	3,49485002	25	125	32	148,413159
6	0,77815125	6	4,6689075	36	216	64	403,428793
7	0,84509804	7	5,91568628	49	343	128	1096,63316
8	0,90308999	8	7,2247199	64	512	256	2980,95799
9	0,95424251	9	8,58818258	81	729	512	8103,08393
10	1	10	10	100	1000	1024	22026,4658



	log(n)	n	n log(n)	n ²	n ³	Exp ⁿ
0	#NOMBRE!	0	#NOMBRE!	0	0	1
5	1,60943791	5	3,49485002	25	125	148,413159
10	2,30258509	10	10	100	1000	22026,4658
15	2,7080502	15	17,6413689	225	3375	3269017,37
20	2,99573227	20	26,0205999	400	8000	485165195
25	3,21887582	25	34,9485002	625	15625	7,2005E+10
30	3,40119738	30	44,3136376	900	27000	1,0686E+13
35	3,55534806	35	54,0423816	1225	42875	1,586E+15
40	3,68887945	40	64,0823997	1600	64000	2,3539E+17
45	3,80666249	45	74,3945631	2025	91125	3,4934E+19
50	3,91202301	50	84,9485002	2500	125000	5,1847E+21
55	4,00733319	55	95,7199479	3025	166375	7,6948E+23
60	4,09434456	60	106,689075	3600	216000	1,142E+26





EXEMPLES

Exemple1

```

Pour i de 1 à n faire
    var = 1
    Tant que var ≤ n faire
        :
        qqe chose en O(1)
        var = cte * var
    Finfaire
Fin pour
  
```

Complexité temporelle?

Exemple 2

```

Pour i de 1 à n faire
    var = n
    Tant que var > 1 faire
        :
        qqe chose en O(1)
        var = var / cte
    Finfaire
Fin pour
  
```

Complexité temporelle ?

La complexité des deux algorithmes est en $O(n \log_{cte}(n))$

Exemple 3 : Tri sélection

Pour les algorithmes de tri la mesure de complexité correspond principalement au nombre de comparaisons des éléments du tableau (avec parfois les permutations).

```

Pour i de 1 à n-1 faire
    min=i
    Pour j de i+1 à n faire
        Si T[j]< T[min] alors
            min=j
    Fin Si
    Finfaire
    Echanger Tab[i] et Tab [min] { x=Tab[i]
                                Tab[i]=Tab[min]
                                Tab[min]=x}
Fin pour

```

Complexité temporelle ?

Complexité temporelle

Pour i =1 la boucle interne effectue n-1 comparaisons
 Pour i =2 la boucle interne effectue n-2 comparaisons
 :
 Pour i =n la boucle interne effectue n-(n-1)=1 comparaisons

Le nombre comparaisons :
 $(n-1)+(n-2)+\dots+1=\sum_{i=1}^{n-1} i = ((n-1)*(1+n-1))/2=(n*(n-1))/2=(n^2/2)-(n/2)$

=>Polynôme de degré 2 Complexité temporelle $O(n^2)$

Signification:

Si on multiplie par 10 le nombre d'éléments à trier
 le temps d'exécution est multiplié par 100

Si on multiplie par 100 le nombre d'éléments à trier
 le temps d'exécution est multiplié par $100^2 (=10000)$

Complexité spatiale $O(n)$

Signification

Si on multiplie par 10 le nombre d'éléments à trier
 le programme utilise un tableau 10 fois plus grand

Si on multiplie par 100 le nombre d'éléments à trier
 le programme utilise un tableau 100 fois plus grand

- L'approximation $n^2/2 - n/2$ se justifie

